

# Beam Element Section Property Tool - Final Report

Subtitle: Functional engineering software deliverable for beam cross-section property calculation Repository: <https://github.com/BESPsoftware/beam-section-property-tool> Technology stack: C++17, CMake, C ABI/shared library, optional Qt Widgets GUI, CTest, GitHub Actions Main deliverables: calculation core, shared-library API, optional GUI, examples, tests, validation material, build scripts, user/developer documentation Date: 2026-06-08 Submission context: Academic/professional software project submission Team/course: Not specified in the repository package

## 1. Executive Summary

Beam Element Section Property Tool is a C++17 engineering software project for modelling beam cross-sections and calculating section properties needed in structural and finite-element workflows. It supports predefined parametric sections and user-defined thin-walled plate sections, computes geometric and torsion-related properties, generates stress output points, provides visualization mesh data, and exports results through several engineering-oriented formats.

The repository contains a complete functional implementation for the documented core/API scope: a reusable calculation core, a C-compatible shared-library interface, optional Qt Widgets GUI, CMake build configuration, example programs, automated tests, numerical validation material, and user/developer documentation. A local no-GUI verification build was executed for this report; the core/API build completed successfully, the aggregated CTest target passed, and all three example programs ran successfully.

The remaining work is bounded and external to the verified core implementation: manual Windows Qt deployment/visual inspection, solver-side acceptance of FEM export files in ANSYS/ABAQUS/Midas Civil, final DOCX manual review, and confirmation of complete construction rules for arbitrary non-reference crane girder geometry.

## 2. Assignment Coverage

| Required area                     | Repository evidence                                                                                                                                                             | Coverage statement                                                                                                                                     | Status                                 |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Requirements and design documents | Documents/source_requirements/, README.md, Documents/Architecture.md, Documents/Implementation_Status.md, Documents/Developer_Guide.md, tests/fixtures/test_data_reference.json | Requirements are derived from source materials, workbook reference data, repository documentation, status records, and executable regression evidence. | Available and evidence-based           |
| Development artifacts             | Source/, Examples/, CMakeLists.txt, build_windows.bat, .github/workflows/cmake.yml                                                                                              | Core calculation, API, GUI, exporters, examples, and build configuration are implemented for the documented project scope.                             | Implemented                            |
| Testing and quality assurance     | tests/, Documents/Test_Report.md, Documents/Numerical_Validation_Report.md, Documents/Final_Acceptance_Checklist.md, Documents/Defect_Log.md, local CMake/CTest run             | Automated core/API regression and integration coverage is present and passed in the local no-GUI verification run.                                     | Implemented and verified               |
| User documentation                | Documents/User Manual.md, Documents/API_Reference.md, Documents/Developer_Guide.md, Documents/Windows_Qt_Verification.md, README.md, Documents/User Manual.docx                 | User, API, developer, and Windows verification documentation is available; DOCX packaging still needs final manual review.                             | Available, final manual review pending |
| External/manual validation        | Documents/Windows_Qt_Verification.md, Documents/Final_Acceptance_Checklist.md, Documents/Test_Report.md                                                                         | Windows Qt deployment and solver-side FEM acceptance are documented as pending manual/external validation steps.                                       | Available, external validation pending |

### 3. Project Overview

The tool is intended for engineers, developers, and project evaluators who need reproducible beam cross-section property calculations. It can be used interactively through the optional Qt GUI or programmatically through the shared-library API declared in `Source/api/section_property_tool.h`.

Supported section inputs are:

- Parametric H section, box section, pipe section, and quayside crane girder definitions.
- User-defined Canvas thin-walled sections supplied as plate centerline segments with thicknesses.

Expected input dimensions are in millimeters. The principal outputs are:

- Section properties: Area,  $J_z$ ,  $J_y$ ,  $J_{yz}$ ,  $J_{zo}$ ,  $J_{yo}$ ,  $J_x$ ,  $A_z$ ,  $A_y$ ,  $c_y$ ,  $c_z$ ,  $\theta$ ,  $C_w$ ,  $y_s$ , and  $z_s$ .
- Default or user-edited stress output points in global and principal coordinates.
- Lightweight visualization mesh data and mesh counts.
- CSV, JSON, ANSYS, ABAQUS, and Midas Civil export files.

The standard user workflow is documented in `Documents/User Manual.md`: choose or create a section, enter geometry, calculate, review properties and graphics, inspect stress points and mesh, and export results when required.

### 4. Requirements and Design Evidence

The repository uses a documentation-driven evidence model rather than a single monolithic requirements specification. Requirements are derived from the original source requirement files, workbook reference data, repository documentation, implementation status records, and tests.

Primary requirement and design evidence includes:

- `Documents/source_requirements/Beam Element Section Property Tool.pdf`
- `Documents/source_requirements/Test Data.xls`
- `tests/fixtures/test_data_reference.json`
- `README.md`
- `Documents/README.md`
- `Documents/Architecture.md`
- `Documents/User Manual.md`
- `Documents/API_Reference.md`
- `Documents/Developer_Guide.md`
- `Documents/Implementation_Status.md`
- `Documents/Final_Acceptance_Checklist.md`

#### Functional Scope

The documented and implemented functional scope includes:

- Build H, box, pipe, quayside crane girder, and Canvas thin-walled sections.
- Validate required geometric parameters and reject invalid input combinations.
- Calculate area, centroid, second moments, product of inertia, principal axes, torsion-related values, shear areas, warping constant, and shear-center coordinates.
- Generate default stress output points and transform points between global and principal coordinates.
- Create lightweight triangular visualization meshes and expose mesh counts through the API.

- Provide a stable C-compatible shared-library interface with opaque handles and explicit destroy/free functions.
- Provide an optional Qt Widgets GUI with General, Stress Points, FE Mesh, and Canvas workflows.
- Export results to CSV, JSON, ANSYS, ABAQUS, and Midas Civil formats.
- Provide examples for parametric, Canvas, and batch API usage.

## Non-Functional Expectations

The repository evidence supports the following non-functional expectations:

- C++17 implementation built with CMake 3.16 or later.
- Core/API build path independent of Qt.
- Optional Qt 5 or Qt 6 GUI selected through SPT\_QT\_VERSION.
- C ABI designed around plain structs, opaque handles, explicit lifecycle management, and thread-local error reporting.
- Core calculation code separated from GUI code.
- UTF-8-aware export path handling on Windows.
- Deterministic regression tests using workbook-derived tolerances.
- No bundled third-party geometry or meshing dependency, as documented in third\_party/README.md.

## Design Assumptions and Constraints

Important design assumptions are explicitly documented:

- All geometry inputs are in millimeters.
- H, box, and pipe reference cases use analytical formulas aligned to the supplied spreadsheet.
- Canvas and non-reference crane girder sections use plate/rectangle representations.
- The supplied crane girder workbook reference case is treated as authoritative because the source PDF diagram does not fully define arbitrary construction coordinates.
- Non-reference crane girder calculations are approximate and emit diagnostics.
- The FE mesh is a visualization mesh, not a solver-quality FEM mesh.
- FEM export writers are implemented and integration-tested for generated content; solver-side acceptance remains an external validation task.

A formal traceability matrix linking each source requirement to code, tests, and documentation would be a useful future documentation improvement, but its absence does not prevent the repository from demonstrating a coherent implemented scope.

## 5. Architecture and Design

Documents/Architecture.md describes a modular architecture with a clear separation between data modelling, geometry construction, calculation, stress points, meshing, export, API, and GUI layers.

The main modules are:

- Source/common: shared data model, section types, result structures, diagnostics, mesh and stress-point data.
- Source/geometry: builders and validation for parametric sections and Canvas plate sections.
- Source/calculation: section property calculation engine.
- Source/stress: stress-point generation, validation, and coordinate transforms.
- Source/mesh: lightweight triangular visualization mesh generation.
- Source/import\_export: CSV, JSON, ANSYS, ABAQUS, and Midas Civil exporters.

- Source/api: C-compatible shared-library wrapper.
- Source/gui: optional Qt Widgets graphical interface.

The documented data flow is:

```
GUI/API input
-> SectionParameters / PlateSegment[]
-> SectionBuilder
-> SectionModel
-> SectionCalculator
-> StressPointEngine
-> MeshEngine
-> GUI/API/Exporter
```

This design supports both interactive use and programmatic integration. The GUI and API share the same core data model and calculation engine, which keeps numerical behavior consistent across usage modes.

## 6. Development Artifacts

The repository contains source code, build configuration, examples, tests, documentation, and platform-specific build support.

### Core Implementation

The primary implementation files are:

- Source/common/DataModel.h
- Source/geometry/SectionBuilder.cpp
- Source/calculation/SectionCalculator.cpp
- Source/stress/StressPointEngine.cpp
- Source/mesh/MeshEngine.cpp
- Source/import\_export/Exporter.cpp
- Source/api/section\_property\_tool.cpp
- Source/api/section\_property\_tool.h
- Source/gui/main.cpp

SectionBuilder validates input and creates SectionModel objects. SectionCalculator computes the section properties. StressPointEngine generates and transforms stress output points. MeshEngine generates visualization meshes. Exporter writes engineering output formats. The C API wraps the core through stable handles and plain C data structures.

### Engineering Calculations

The calculation implementation supports:

- Analytical calculations for H sections, box sections, and pipe sections.
- Composite rectangle calculations for plate-based sections.
- Principal inertia values and principal-axis angle from the centroidal inertia tensor.
- Workbook-aligned pipe shear-area factor.
- Open-section torsion approximation based on plate length and thickness.
- H-section warping constant and centroidal shear center.
- Zero warping constant for symmetric closed box and pipe sections.
- Numerical open thin-walled sectorial-area integration for crane girder and Canvas sections.

For the exact crane girder reference parameter set, the builder stores workbook-derived reference values. For other crane parameter sets, the software builds an approximate named plate graph and reports a diagnostic warning.

## API and Shared Library

The shared-library target is `SectionPropertyApi`; the output name is `SectionPropertyTool`. The public API in `Source/api/section_property_tool.h` exposes:

- Version query.
- Section creation from parametric values or Canvas plate lines.
- Property calculation.
- Property, stress-point, and mesh-count retrieval.
- Stress-point updates.
- CSV, JSON, ANSYS, ABAQUS, and Midas Civil export.
- Explicit cleanup functions for sections, results, meshes, and stress-point arrays.
- Thread-local error retrieval through `spt_get_last_error()`.

`Documents/API_Reference.md` provides handle lifecycle rules, units, enum values, function descriptions, and a minimal usage example.

## Optional GUI

The Qt Widgets GUI in `Source/gui/main.cpp` provides:

- General tab for section selection, parameter editing, property table, and section preview.
- Stress Points tab for reviewing and editing output points.
- FE Mesh tab for visualization mesh display and refinement control.
- Canvas tab for plate table editing, graphical drawing, selection, endpoint dragging, grid display, snap-to-grid, and Canvas section calculation.

The GUI is optional. If Qt Widgets is not found, CMake still builds the core library, shared API, examples, and tests.

## Build and Configuration

`CMakeLists.txt` defines the principal build targets:

- `SectionPropertyCore`
- `SectionPropertyApi`
- `SectionPropertyTests`
- `example1_parametric`
- `example2_canvas`
- `example3_dll_batch`
- `SectionPropertyGui`, when Qt Widgets is available
- `SectionPropertySmokeTest`, when Qt Widgets and Qt Test are available

`build_windows.bat` documents and automates the Windows Qt build/deployment path for supported Qt installations. `.github/workflows/cmake.yml` builds and tests the no-GUI path on Ubuntu, macOS, and Windows.

## 7. Testing and Quality Assurance

Testing and QA evidence is provided through automated C++ tests, validation reports, source reference data, status documents, an acceptance checklist, defect tracking, and CI configuration.

### Automated Test Structure

The aggregated CTest target is `SectionPropertyTests`, built from:

- `tests/unit/test_main.cpp`
- `tests/regression/test_regression.cpp`

- tests/integration/test\_api.cpp

The target includes regression checks for workbook-derived section properties, Canvas validation behavior, non-reference crane girder diagnostics, API smoke coverage, mesh counts, export content checks, empty stress-point export behavior, and UTF-8 export paths.

When Qt Widgets and Qt Test are available, tests/smoke/windows\_gui\_smoke\_test.cpp is built as SectionPropertySmokeTest. That target constructs the GUI offscreen, switches section types, verifies property/stress/mesh state, and builds the documented Canvas example section.

## Numerical Validation

Documents/Numerical\_Validation\_Report.md compares expected and actual values from Documents/source\_requirements/Test Data.xls, with the workbook data also represented in tests/fixtures/test\_data\_reference.json. The documented validation covers:

- H section.
- Hollow box section.
- Pipe section.
- Supplied quayside crane girder reference case.
- Warping constant and shear-center behavior for supported section categories.

The report records pass results within the tolerances encoded in the regression suite.

## Local Verification Performed for This Report

The following commands were executed in a safe no-GUI configuration:

```
cmake -S . -B build-final-report -DCMAKE_BUILD_TYPE=Release -DSPT_QT_VERSION=OFF
cmake --build build-final-report --config Release
ctest --test-dir build-final-report --output-on-failure
./build-final-report/Library/bin/example1_parametric
./build-final-report/Library/bin/example2_canvas
./build-final-report/Library/bin/example3_dll_batch
git diff --check
```

Observed outcome:

- CMake configured successfully with AppleClang.
- Qt Widgets were not used because SPT\_QT\_VERSION=OFF; core/API targets, tests, and examples remained enabled.
- Core static library and C API shared library built successfully.
- SectionPropertyTests built successfully.
- CTest executed the aggregated core/API regression target successfully.
- example1\_parametric reported H-section area 6520 mm<sup>2</sup>, C<sub>w</sub> = 3.00833e+10 mm<sup>6</sup>, and shear center (50, 125) mm.
- example2\_canvas reported Canvas area 2960 mm<sup>2</sup> and centroid (50, 60) mm.
- example3\_dll\_batch reported successful H-section and pipe-section batch calculations.
- git diff --check reported no whitespace errors.

This verification confirms the no-GUI core/API build and executable examples in the local environment. It does not replace the separate Windows Qt deployment checklist or external solver acceptance.

## QA and Acceptance Documents

Additional QA material is available in:

- Documents/Test\_Report.md
- Documents/Numerical\_Validation\_Report.md

- Documents/Final\_Acceptance\_Checklist.md
- Documents/Windows\_Qt\_Verification.md
- Documents/Defect\_Log.md
- .github/workflows/cmake.yml

Documents/Defect\_Log.md records closed issues related to calculation, geometry, export, integration, and GUI Canvas editing. The final acceptance checklist records completed core/API evidence and explicitly identifies manual/external validation tasks still pending.

## 8. User and Developer Documentation

The repository includes documentation for users, API consumers, developers, and platform-specific verification.

### User Documentation

Documents/User Manual.md describes:

- General tab workflow.
- Section parameters.
- Stress point editing.
- FE mesh visualization.
- Canvas plate input and validation.
- Export behavior.
- Validation notes and common issues.

The user manual explains that geometry inputs are in millimeters, calculations use double precision internally, GUI tables may round values, stress points are output locations rather than mesh nodes, and the FE mesh is intended for visualization.

### API Documentation

Documents/API\_Reference.md documents:

- Shared library/header information.
- Handle lifecycle.
- Section type enums.
- Required parameters.
- Canvas input structs.
- Calculation, property, stress-point, mesh, and export functions.
- Error handling.
- Units.
- Minimal example code.

Documents/DLL\_Interface.md also describes the handle model and thread-safety note. Its export-format note is older than the current API\_Reference.md and source implementation, so it should be reconciled during documentation maintenance.

### Developer Documentation

Documents/Developer\_Guide.md documents prerequisites, repository structure, CMake targets, build commands, test commands, examples, extension steps for new section types and export formats, coding conventions, and troubleshooting guidance.

README.md provides a project overview, features, supported section types, quick-start commands, platform build notes, validation status, examples, documentation index, and remaining validation boundaries.

## Platform Documentation

Documents/Windows\_Qt\_Verification.md defines the Windows Qt 5.15.2 GUI build, deployment, expected artifacts, exported symbol verification, Qt deployment, automated smoke coverage, and manual runtime checklist.

## 9. Deliverables Summary

| Assignment category               | Supporting artifacts                                                                                                                                                        | Delivery status                                                             |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Requirements and design documents | Source requirement archive, workbook-derived fixture, architecture notes, README, implementation status, developer guide                                                    | Available as distributed evidence                                           |
| Development artifacts             | Core source modules, C API, optional Qt GUI, exporters, examples, CMake configuration, Windows build script, CI workflow                                                    | Implemented for documented scope                                            |
| Testing and quality assurance     | Aggregated CTest target, regression tests, API integration tests, GUI smoke test target when Qt is available, numerical validation report, acceptance checklist, defect log | Core/API verified locally; GUI and solver validations documented separately |
| User documentation                | User manual, API reference, developer guide, Windows Qt verification guide, README, DOCX manual package                                                                     | Available; DOCX final review pending                                        |

The core implementation is complete for the documented project scope and the local no-GUI verification confirms the core/API build, aggregated regression target, and examples. GUI support is optional and Qt-dependent. Pending Windows Qt visual deployment and solver-side FEM acceptance are manual/external validation activities; they do not indicate absence of the implemented core functionality.

## 10. Validation Boundaries and Future Work

The project has clear validation boundaries. These items are controlled next steps rather than evidence of missing core implementation:

- Windows Qt deployment: final windeployqt packaging and visual/manual runtime inspection should be completed on an interactive Windows machine with the documented Qt installation.
- FEM solver acceptance: ANSYS, ABAQUS, and Midas Civil writers are implemented and integration-tested for generated content, but acceptance should be confirmed inside the target solver applications.
- Non-reference crane girder geometry: arbitrary crane girder parameter sets use approximate plate-graph behavior until complete construction rules are confirmed or formally accepted.
- DOCX user manual package: Documents/User Manual.docx is available, but final formatting and packaging review remains pending.
- Documentation maintenance: older notes such as parts of Documents/DLL\_Interface.md and tests/gui\_smoke/README.md should be reconciled with the current API/export and GUI smoke-test implementation.
- Requirements traceability: a formal traceability matrix would strengthen future submissions by linking source requirements, implementation modules, tests, and documentation.
- Mesh capability: the current mesh is a visualization mesh; a solver-quality meshing backend would be future work if solver mesh export becomes a requirement.

## 11. Conclusion

Beam Element Section Property Tool is suitable for submission as a functional engineering software deliverable. The repository provides a C++17 calculation core, C-compatible shared-library API, optional Qt GUI, build scripts, example programs, automated tests,

numerical validation material, and user/developer documentation.

The local verification performed for this report confirms the core/API no-GUI build, aggregated CTest regression target, and example workflows. Remaining work is appropriately framed as external validation and future hardening: Windows Qt deployment inspection, solver-side FEM acceptance, DOCX final review, documentation reconciliation, and confirmation of arbitrary crane girder construction rules.